1. Loft Language Reference

A statically-typed scripting language inspired by Rust, designed for embedding in the Dryopea engine.

Every code example in this document is an executable part of the test suite.

Contents

1. Loft Language Reference ... 1
2. vs Rust ... 5
   - 2.0.1. Variables — no let or mut ... 5
   - 2.0.2. Null instead of Option<T> ... 5
   - 2.0.3. Parameter mutability — const and & instead of ownership ... 6
   - 2.0.4. ^ is exponentiation, not XOR ... 6
   - 2.0.5. No loop or while ... 7
   - 2.0.6. Filtered loops — for ... if 7
   - 2.0.7. Loop attributes: #first, #count, #index ... 7
   - 2.0.8. Named loop break — x#break ... 8
   - 2.0.9. Methods by self name, not impl blocks ... 8
   - 2.0.10. Polymorphic enum dispatch ... 9
   - 2.0.11. No closures — but compile-checked function references ... 9
   - 2.0.12. No generic functions or trait system ... 10
   - 2.0.13. String formatting — embedded expressions ... 11
   - 2.0.14. Built-in parallel for-loops — par(...) ... 11
3. Keywords ... 13
   - 3.0.1. Conditionals ... 13
   - 3.0.2. Iteration ... 13
   - 3.0.3. Nested Loops and Break ... 14
   - 3.0.4. Loop Helpers: #first and #count ... 14
   - 3.0.5. Formatting Iterations ... 15
4. Texts ... 16
   - 4.0.1. Creating and Measuring Strings ... 16
   - 4.0.2. Iterating Over Characters ... 17
   - 4.0.3. Searching and Splitting ... 17
   - 4.0.4. Text Formatting ... 17
5. Integers ... 18
6. Boolean ... 20
7. Float ... 21
   - 7.0.1. Formatting Floats ... 21
   - 7.0.2. Math Functions ... 21
8. Functions ... 23
   - 8.0.1. Declaring Functions ... 23
   - 8.0.2. Reference Parameters and Defaults ... 23
   - 8.0.3. Early Return ... 23
   - 8.0.4. Const and Reference Parameters ... 23
   - 8.0.5. Type-Based Dispatch ... 23
9. Vector ... 25
   - 9.0.1. Slicing ... 25
   - 9.0.2. Comprehensions ... 25
   - 9.0.3. Reverse Iteration ... 26

10. Structs 27
10.0.1. Field Constraints 27
10.0.2. Methods 27
10.0.3. Computed Fields 27
10.0.4. Formatting Structs 28
11. Enums 29
11.0.1. Struct Enums 29
11.0.2. Polymorphic Dispatch 29
12. Sorted 31
12.0.1. Adding Elements 31
12.0.2. Looking Up by Key 31
12.0.3. Iterating in Order 31
12.0.4. Loop Helpers: #first, #count, and #remove 32
12.0.5. Removing by Key 32
13. Index 34
13.0.1. Adding and Looking Up Records 34
13.0.2. Iterating in Order 34
13.0.3. Range Queries 34
13.0.4. Loop Helpers: #first and #count 35
13.0.5. Removing by Key 35
14. Hash 36
14.0.1. Combining Hash and Vector 36
14.0.2. Basic Lookup 36
14.0.3. Hash + Vector Together 36
14.0.4. Removing a Key 37
14.0.5. Why you cannot iterate a hash directly 37
15. File 38
15.0.1. Inspecting the File System 38
15.0.2. Writing a Text or Binary File 38
15.0.3. Reading Back What You Wrote 39
15.0.4. Working with Vectors and Struct Data 40
16. Image 42
16.0.1. Loading an Image 42
16.0.2. Reading Dimensions 42
16.0.3. Accessing Individual Pixels 42
16.0.4. Iterating Over All Pixels 42
17. Lexer 43
17.0.1. Setting Up the Lexer 43
17.0.2. Reading Tokens 43
17.0.3. String Literals and Comments 43
17.0.4. Embedded Format Expressions 44
18. Parser 45
19. Libraries 46

19.0.1. Constants and Enums ... 46
19.0.2. Struct Construction and Field Access ... 46
19.0.3. Calling Library Methods ... 46
19.0.4. Extending a Library Type ... 46
19.0.5. Limitations ... 48
20. Store Locks ... 49
20.0.1. Checking and Setting a Lock ... 49
20.0.2. The const Keyword for Local Variables ... 49
20.0.3. Passing const to a Function ... 50
21. Parallel execution ... 51
21.0.1. Running a Global Function in Parallel ... 52
21.0.2. Running a Method in Parallel ... 53
22. Logging ... 54
23. Standard Library ... 55
23.1. Types ... 55
23.2. Math ... 56
23.3. Text ... 58
23.4. Collections ... 61
23.5. Output and Diagnostics ... 61
23.6. Parallel ... 62
23.7. File System ... 62
23.8. Environment ... 64

2. vs Rust

Loft is inspired by Rust but designed for embedded scripting in the Dryopea engine. It trades some of Rust's safety guarantees and expressive power for a shorter learning curve and less ceremony. This page documents the most important differences so that Rust programmers know exactly what they gain and what they give up.

2.0.1. Variables — no let or mut

```
x = 42
x += 1
const y = 42   // opt-in: locked in debug builds
```

```
let mut x = 42;
x += 1;
let y = 42;     // immutable by default; compiler-enforced
```

Upside Less boilerplate. No need to decide upfront whether a variable will be mutated. Variables are mutable by default, so refactoring is frictionless. Use const to signal that a value should not change — in debug builds the runtime locks the store immediately after initialisation, catching accidental writes early.

Downside Immutability is opt-in, not the default. Rust makes variables immutable unless you write mut, catching accidents at compile time for free. Loft's const lock is only checked at runtime and only in debug builds, so it provides less of a safety net.

2.0.2. Null instead of Option<T>

```
struct Point {
    label: text,          // nullable
    x: integer not null,  // never null
}
p = Point { x: 1 };
assert(p.label == null, "absent");
```

```
struct Point {
    label: Option<String>,  // explicit absence
    x: i32,                 // always present
}
let p = Point { label: None, x: 1 };
assert!(p.label.is_none());
```

Upside Natural for database records where fields are often absent. No Some(x)/None wrapping. Conditionals on null are concise: if v == null.

Downside Null is opt-out, not opt-in. A field is nullable unless explicitly marked not null, so forgetting the annotation leaves a potential null silently. In Rust, Option is visible in the type and forces handling at every call site.

2.0.3. Parameter mutability — const and & instead of ownership

```
// Structs/collections: always by reference
fn push_two(v: vector<integer>) {
    v += [1, 2];  // caller sees the change
}
// const: read-only (locked in debug builds)
fn count(v: const vector<integer>) - integer {
    length(v)
}
// &: explicit write-back for primitives
fn add_to(n: &integer, delta: integer) {
    n += delta;
}
```

```
fn push_two(v: &mut Vec<i32>) {
    v.push(1);
    v.push(2);
}
fn count(v: &Vec<i32>) -> usize {
    v.len()
}
fn add_to(n: &mut i32, delta: i32) {
    *n += delta;
}
```

Upside No borrow-checker errors. No lifetime annotations. No ownership transfer to reason about. The model is simpler: structs and collections are always references, primitives are copies unless marked & (explicit write-back). const parameters signal read-only intent, and in debug builds the runtime locks the store for the duration of the call — enough to catch most accidents during development.

Downside The compile-time guarantees are much weaker. Rust's borrow checker statically proves no aliased mutations, no dangling references, and no data races — before the program ever runs. Loft's const is a debug-only runtime check. Aliasing is unchecked at compile time, and the engine's Rust runtime is what truly enforces memory safety.

2.0.4. ^ is exponentiation, not XOR

```
area = PI * (r ^ 2.0)   // exponentiation
bits = a | b              // bitwise OR
mask = a & b              // bitwise AND
```

```
let area = PI * r.powi(2); // no ^ operator
let bits = a | b;          // bitwise OR
let mask = a & b;          // bitwise AND
let xor  = a ^ b;          // bitwise XOR — no loft equivalent
```

Upside x^2 reads like mathematical notation. No need for .powi() or .powf() method calls. The library functions pow(x, y) also exist for the free-function style.

Downside Rust developers expect ^ to be XOR; the adjustment takes time. ^ has lower precedence than *, so PI * r^2 must be written PI * (r^2) — easy to forget. Bitwise XOR has no operator in loft.

2.0.5. No loop or while

```
for _ in 0.. {          // open-ended range
    if done() { break; }
    step();
}
```

```
loop {
    if done() { break; }
    step();
}
while !done() { step(); }
```

Upside Every loop has an iteration variable, making it easy to add index tracking or break conditions without restructuring. Open-ended for _ in 0.. is still possible when truly unbounded iteration is needed.

Downside while condition { } is universally understood and directly expresses intent. Its absence surprises Rust and C programmers. Event loops and polling patterns are more verbose with the open-ended range workaround.

2.0.6. Filtered loops — for … if …

```
for x in items if x > 0 {
    total += x;
}
```

```
for x in items.iter().filter(|&&x| x > 0) {
    total += x;
}
```

Upside Reads like natural language. No closure syntax, no double-reference in the filter predicate. v#remove inside a filtered loop also safely removes the current element while iterating — something that requires careful index management in Rust.

Downside Cannot compose with other adapters. Rust's iterator chain (.filter().map().take_while()) is far more expressive for complex data pipelines. Each extra step in loft requires a new loop or a helper variable.

2.0.7. Loop attributes: #first, #count, #index

```
for x in 1..=5 {
    if !x#first { b += ", "; }
    b += "{x#count}:{x}";
}
```

```
for (i, x) in (1..=5).enumerate() {
    if i != 0 { b.push_str(", "); }
    b.push_str(&format!("{i}:{x}"));
}
```

Upside No tuple destructuring needed. x#first reads as "is this the first x", which is self-explanatory. Eliminates helper counter variables for the common pattern of comma-separated output.

Downside The #attribute syntax is unique to loft and unfamiliar to everyone else. Rust's .enumerate() composes freely with other adapters; loft's attributes are only available on the loop variable of the innermost loop.

2.0.8. Named loop break — x#break

```
for x in 1..5 {
    for y in 1..5 {
        if x * y >= 6 { x#break; }
    }
}
```

```
'outer: for x in 1..5 {
    for y in 1..5 {
        if x * y >= 6 { break 'outer; }
    }
}
```

Upside Reuses the existing loop variable name as the label. No separate 'label declaration at the loop head. The name x#break reads as "break out of the loop over x".

Downside Rust's lifetime-label syntax 'outer is explicit and visually separate from the loop body. x#break is easy to confuse with the loop attributes x#first and x#count, which have completely different semantics.

2.0.9. Methods by self name, not impl blocks

```
fn greet(self: Person) - text {
    "Hello, {self.name}!"
}
fn name_len(self: const Person) - integer {
    length(self.name)  // const: read-only access guaranteed
}
p.greet()     // dot syntax
greet(p)      // free-function call — identical
```

```
impl Person {
    fn greet(&self) -> String {
        format!("Hello, {}!", self.name)
    }
    fn name_len(&self) -> usize {
```

```
        self.name.len()
    }
}
p.greet();     // only dot syntax
```

Upside No impl block ceremony. Methods and free functions are the same thing — just a naming convention. Functions can be added to any type from any file without modifying the original struct definition, similar to extension methods. Mark the first parameter const to guarantee the method never modifies the receiver — analogous to Rust's &self.

Downside No consuming self. Plain (non-const) methods behave like &mut self — the caller's value is always mutably aliased. Multiple functions with the same name but different non-variant self types are a compile error — no standard overloading.

2.0.10. Polymorphic enum dispatch

```
enum Shape {
    Circle { r: float },
    Rect   { w: float, h: float }
}
fn area(self: Circle) - float { PI * (self.r ^ 2.0) }
fn area(self: Rect)   - float { self.w * self.h }

s.area()    // dispatches on the runtime variant
```

```
enum Shape {
    Circle { r: f64 },
    Rect   { w: f64, h: f64 },
}
fn area(s: &Shape) -> f64 {
    match s {
        Shape::Circle { r } => PI * r * r,
        Shape::Rect { w, h } => w * h,
    }
}
```

Upside Each variant's behaviour lives in its own small function — easy to read and extend. Adding a new type of shape only requires a new fn area(self: NewShape) without modifying the dispatch site. Feels like OOP method overriding without the inheritance.

Downside Rust's match is exhaustive: the compiler forces you to handle every variant. In loft, adding a new variant silently leaves area unimplemented for it — the call will panic at runtime. Correctness relies on discipline, not the compiler.

2.0.11. No closures — but compile-checked function references

```
// fn <name> produces a compile-time-checked function reference
fn double(r: const Score) - integer { r.value * 2 }
```

```
// Pass it to par(...) for parallel dispatch:
for item in scores par(result=double(item), 4) {
    total += result;
}

// Context must be embedded in the element — no captures:
fn apply(v: vector<integer>, offset: integer) {
    for x in v { x += offset; }
}
```

```
fn double(r: &Score) -> i32 { r.value * 2 }

// Rayon parallel iterator:
let total: i32 = scores.par_iter().map(double).sum();

// Closure captures context freely:
let offset = 5;
let shifted: Vec<i32> = v.iter()
    .map(|x| x + offset)
    .collect();
```

Upside Simpler mental model. No capture modes (move vs borrow), no Fn/FnMut/FnOnce trait bounds, no lifetime constraints on captured variables. The fn <name> expression gives a compile-checked reference to any named function — the compiler verifies the name exists and has a matching signature before emitting any code.

Downside No general higher-order functions. fn <name> references are only consumed by the par(...) parallel-loop clause; they cannot be stored in a variable or passed to arbitrary user code. Callbacks that need local context must embed everything in a struct — there is no way to capture a surrounding variable the way a Rust closure would.

2.0.12. No generic functions or trait system

```
// No generic functions — must write a version per type
fn max_int(a: integer, b: integer) - integer {
    if a  b { a } else { b }
}
fn max_float(a: float, b: float) - float {
    if a  b { a } else { b }
}
```

```
fn max<T: PartialOrd>(a: T, b: T) -> T {
    if a > b { a } else { b }
}
```

Upside Nothing to learn about type parameters, trait bounds, where clauses, dyn Trait, impl Trait, or associated types. Collections (vector<T>, hash<T>, sorted<T>) are generic at the engine level, covering the most common need.

Downside Code cannot be written once and reused across types. Every generic algorithm must be duplicated per type or pushed down into the standard library. There is no way for user code to define an “anything sortable” or “anything printable” abstraction.

2.0.13. String formatting — embedded expressions

```
msg  = "Hi {name}, score: {score:+6.2}"  // width.prec
prec = "{value:.2}"                      // precision only
hex  = "{n:#x}"
list = "{for x in 1..4 {x*2}}"          // "2,4,6"
// Nested quotes in format expressions require a variable:
hw = "hello"; "{hw:3}"                  // not {\"hello\":3}
```

```
let msg  = format!("Hi {name}, score: {:+6.2}", score);
let prec = format!("{:.2}", value);
let hex  = format!("{:#x}", n);
// no in-string iteration; needs a separate collect step
// nested string literals in format args are fine in Rust
```

Upside Concise — no format!() call, no separate variable. Full format expressions (arithmetic, slices, for comprehensions) can appear directly inside {}. Specifiers mirror Rust: :width, :.precision, :width.precision, sign (+), radix (#x, #o, b), alignment (<, >, ^), and zero-padding (0N) all work.

Downside No compile-time validation of format strings — typos in specifiers are runtime errors. Nested string literals inside a {...} format expression cause a parse error (work around: store the string in a variable first). Zero-padding a negative integer places padding before the sign ({-1:03} ⟶ 0-1), which differs from what most programmers expect.

2.0.14. Built-in parallel for-loops — par(...)

```
fn double(r: const Score) - integer { r.value * 2 }

sum = 0;
for item in scores par(b=double(item), 4) {
    sum += b;   // b is double(item), computed in parallel
}               // results arrive in original order

// Method form:
for item in scores par(b=item.value(), 4) { ... }
```

```
use rayon::prelude::*;

fn double(r: &Score) -> i32 { r.value * 2 }

let sum: i32 = scores.par_iter()
    .map(double)
    .sum();      // order is not guaranteed without extra work

// rayon is an external crate; add to Cargo.toml first
```

Upside Built into the language — no external crate, no Cargo.toml edit. The compiler validates the worker function signature at the call site. Results are delivered in the original vector order. The thread count is set per call, making it easy to tune for the hardware at hand.

Downside The worker must return a primitive (integer, long, float, or boolean) — returning text or a struct reference is not yet supported. Workers cannot capture local variables (all context must be embedded in the element struct). Iterator-chain composition (filter, reduce, scan) is not available; each stage needs a separate loop.

3. Keywords

This page walks through the basic building blocks of Loft: conditions, loops, and breaks. Every example is checked with an assert so you can see what the result should be.

```
fn main() {
```

3.0.1. Conditionals

Use 'if' to run code only when a condition holds. 'panic' stops the program immediately and logs the message. In finished software you would handle the problem differently, but during development panic is a quick way to catch impossible situations.

```
    if 2 > 5 {
       panic("Incorrect test");
    }
```

You can combine conditions with 'and' and 'or' (or their symbol equivalents '&&' / '||'). '&' is the bitwise AND operator and works on the raw bits of a number.

```
    a = 12;
    if a > 10 and a & 7 == 4 {
        a += 1;
    } else {
        a -= 1;
    }
```

Text enclosed in double quotes may contain expressions inside curly braces. The expression is evaluated and inserted into the text at that position. 'assert' checks that its first argument is true. If it is false the message is logged.

```
    assert(a == 13, "Incorrect value {a} != 13");
```

In Loft, every block of code produces a value — its last expression. That means you can use 'if/else' on the right-hand side of an assignment.

```
    b = if a == 13 { "Correct" } else { "Wrong" };
    assert(b == "Correct", "Logic expression");
```

3.0.2. Iteration

'for x in a..b' loops over the integers from a up to but not including b.

```
    t = 0;
    for a in 1..6 {
        t += a;
    }
    assert(t == 15, "Total was {t} instead of 15");
```

‘rev(range)’ reverses the order of a range. ‘1..=5’ is an inclusive range: it includes 5 as the last value.

```
    t = 0;
    for a in rev(1..=5) {
        t = t * 10 + a;
    }
    assert(t == 54321, "Result was {t} instead of 54321");
```

3.0.3. Nested Loops and Break

Loft has no ‘while’ or ‘loop’ keyword. All repetition uses ‘for’ with a range. This keeps programs predictable — there is always a clear upper bound on iterations. ‘break’ exits the nearest enclosing loop. To exit an outer loop from inside an inner one, prefix the loop variable with ‘#break’. ‘len(s)’ returns the number of bytes in a text value.

```
    b = "";
    for x in 1..5 {
        for y in 1..5 {
            if y > x {
                break; // this breaks the inner y loop
            }
            if x * y >= 16 {
                x#break;
            }
            if len(b) > 0 {
                b += "; ";
            }
            b += "{x}:{y}";
        }
    }
    assert(b == "1:1; 2:1; 2:2; 3:1; 3:2; 3:3; 4:1; 4:2; 4:3", "Incorrect
sequence '{b}'");
```

3.0.4. Loop Helpers: #first and #count

Inside a loop body you can ask for extra information about the current iteration. ‘x#first’ is true only for the first element that passes the optional filter. ‘x#count’ is a running index that starts at 0 and goes up by one for each element kept. An ‘if’ clause after ‘in range’ filters which elements enter the loop.

```
    b = "";
    for x in 1..=9 if x % 3 != 1 {
        if !x#first {
            b += ", ";
        }
        b += "{x#count}:{x}";
    }
    assert(b == "0:2, 1:3, 2:5, 3:6, 4:8, 5:9", "Sequence '{b}'");
```

3.0.5. Formatting Iterations

You can embed a for loop directly inside a format string. The results are collected into a bracketed, comma-separated list. A format specifier after ':' is applied to every element in the list.

```
    assert("a{for x in 1..7 {x*2}:02}b" == "a[02,04,06,08,10,12]b", "Format
range");
}
```

4. Texts

Text in Loft is stored as UTF-8, the same encoding used on the web. Most operations work in bytes rather than characters, which is fast but requires care when your text contains multi-byte characters like emoji.

```
fn main() {
```

4.0.1. Creating and Measuring Strings

Join two pieces of text with '+'. The result is a new text value.

```
    a = "1" + "2";
    assert("'{a:4}'" == "'12  '", "Formatting text");
    assert(len(a + "123") == 5, "Text length");
```

'len()' counts bytes, not visible characters. An emoji like '😃' takes 4 bytes; a special symbol like '♥' takes 3 bytes. Plain ASCII letters each take exactly 1 byte.

```
    assert(len("😃") == 4, "Emoji byte length");
    assert(len("♥") == 3, "Heart byte length");
    assert(len("abc") == 3, "ASCII byte length");
```

You can read a single character using a byte index in square brackets. For plain ASCII text every byte holds one character, so this is straightforward.

```
    s = "ABCDE";
    assert(s[0] == 'A', "Character at byte 0");
    assert(s[2] == 'C', "Character at byte 2");
```

A slice like 's[1..4]' gives you the bytes from index 1 up to but not including 4. Leaving out the start ('s[..n]') is the same as starting at 0. Leaving out the end ('s[n..]') goes all the way to the last byte.

```
    assert(s[0..2] == "AB", "Explicit start slice");
    assert(s[..2] == "AB", "Open-start slice");
    assert(s[1..4] == "BCD", "Sub-string by byte range");
    assert(s[3..] == "DE", "Open-ended sub-string");
```

When the text contains multi-byte characters, Loft snaps a byte offset to the nearest character boundary so you never split a character in two. A negative end index counts from the end: −1 means "stop one byte before the end".

```
    txt = "12😊🙃45";
    assert(txt[2..-1] == "😊🙃4", "UTF-8 sub-string by byte range");
```

4.0.2. Iterating Over Characters

A for loop over a text value visits one Unicode character at a time. '#index' is the byte position where the current character starts. '#next' is the byte position right after the current character ends. For "Hi 😊!": H starts at 0, i at 1, space at 2, the emoji at 3 (ends at 7), ! at 7.

```
    result = "";
    positions = [];
    nexts = [];
    for c in "Hi 😊!" {
        positions += [c#index];
        nexts += [c#next];
        if c#index >= 3 {
            result += c;
        }
    }
    assert(result == "😊!", "Character iteration was {result}");
    assert("{positions}" == "[0,1,2,3,7]", "Character positions was
{positions}");
    assert("{nexts}" == "[1,2,3,7,8]", "Character nexts was {nexts}");
```

4.0.3. Searching and Splitting

These built-in functions let you look for text inside other text. All positions returned are byte offsets.

```
    assert("something".starts_with("some"), "starts_with");
    assert("something".ends_with("thing"), "ends_with");
    assert("something".find("th") == 4, "find returns byte offset");
    assert("a longer text".contains("longer"), "contains");
```

'trim()' removes spaces and other whitespace from both ends of a text value.

```
    assert(trim("  hello  ") == "hello", "trim");
```

4.0.4. Text Formatting

Inside a format string, '{' and '}' are special characters. Write '{{' to get a literal '{' and '}}' to get a literal '}'.

```
    brace_inner = "cd";
    assert("ab{{cd}}e" == "ab{{{brace_inner}}}e", "Escaping braces");
```

You can control how a value is aligned inside a fixed-width field. '<' means left-align, '>' means right-align (the default). The width after the ':' can be a constant or a small expression.

```
    vr = "abc";
    assert("1{vr:<2+3}2{vr}3{vr:6}4{vr:>7}" == "1abc  2abc3abc   4    abc", "Text
alignment");
}
```

5. Integers

The default number type in Loft is 'integer', a 32-bit signed whole number (the same range as i32 in Rust: roughly −2 billion to +2 billion). This page shows arithmetic, conversions, bitwise operations, and what happens when something goes wrong — like dividing by zero.

```
fn main() {
```

Numbers and text can be converted back and forth. Wrapping a value in curly braces inside a string formats it as text. 'as integer' parses a text value into a number.

```
    v = 4;
    assert("{v}" == "4", "Convert numbers to text");
    assert("123" as integer == 123, "Convert text to number");
```

Loft follows standard operator precedence: multiply before add, and so on. Bitwise operators like '<<' (left shift), '&' (bitwise AND), and '^' (XOR) also follow a defined precedence — when in doubt, use parentheses.

```
    assert(1 + 2 * 4 == 9, "Integer calculus");
    assert(1 + 2 << 2 == 12, "Binary shift");
    assert(0x0a8 & 15 == 8, "Binary and");
    assert(42 ^ 0b111111 == 21, "Binary xor");
    assert(105 % 100 == 5, "Modulus");
    assert(pow(2.0, 3.0) == 8.0, "Power");
```

'abs()' is a regular function, not a method on a number.

```
    assert(1 + abs(-2) == 3, "Absolute integer");
```

Dividing by zero does not crash the program. Instead it produces a special 'null' value that behaves like false in a condition.

```
    a = 2 * 2;
    a -= 4;
    assert(!(12 / a), "Null on division by zero");
```

```
    assert("a{12}b" == "a12b", "Formatting problem");
```

Format strings can contain full expressions, not just variable names. '#x' formats the result as a hexadecimal number with a '0x' prefix.

```
    assert("a{1 + 2 * 3:#x}b" == "a0x7b", "Hex formatting");
```

Hexadecimal literals can use either lowercase (a–f) or uppercase (A–F) digits.

```
    assert(0xff == 255, "Lowercase hex literal");
    assert(0x2a == 42, "Lowercase hex literal 2");
    assert(0xFF == 255, "Uppercase hex literal");
    assert(0x2A == 42, "Uppercase hex literal 2");
```

Format specifiers give you control over how a number is displayed. '#x' = hex with prefix, 'o' = octal, 'b' = binary, '+' = always show a sign, '0N' = zero-pad to N digits wide.

```
    assert("ab{1+2+32:#x}c{12:o}d{391}e{12:+4}f{1:03}g{42:b}h" == "ab0x23c14d391e
+12f001g101010h", "Integer formats");
```

The 'long' type (suffix 'l') holds 64-bit integers, useful when values exceed the 32-bit range. Use 'as integer' to convert a long back to a regular integer.

```
    assert("a{1l+1:+4}b{12l as integer}c {2l * (4l % 6l) >= 8} d" == "a  +2b12c
true d", "Long formatting");
}
```

6. Boolean

A boolean is either ‘true’ or ‘false’. Loft also has a third state called ‘null’, which means “no value”. In a condition, null behaves like false. This page covers logical operators, bitwise operators, and null.

```
fn main() {
    assert(!(3 > 2 + 4), "Boolean test");
    assert(true as text == "true", "Convert boolean to text");
```

‘and’ and ‘or’ are the same as ‘&&’ and ‘||’ — pick whichever reads better. Both use short-circuit evaluation: if the left side already decides the result, the right side is never evaluated.

```
    assert(1 > 0 and 2 > 1, "Both conditions true");
    assert(1 > 2 or 3 > 2, "Second condition true");
    assert(!(1 > 2 && 3 > 2), "First false, whole 'and' is false");
    assert(!(1 > 2 || 3 > 4), "Both false, whole 'or' is false");
```

Some operations can produce ‘null’ instead of a number. Dividing by zero is the simplest example. In a boolean context, null is treated as false.

```
    zero = 0;
    assert(!(12 / zero), "Null from division-by-zero is false");
    assert(12 > 0, "Positive integer is true");
```

Bitwise operators work on the individual bits of an integer. ‘&’ keeps only bits that are set in both operands. ‘|’ keeps bits that are set in either operand. ‘<<’ shifts bits to the left, effectively multiplying by powers of two. ‘>>’ shifts bits to the right, effectively dividing by powers of two. Because ‘&’ binds less tightly than comparison operators, use parentheses when mixing them.

```
    assert((0x0f & 0xa8) == 8, "Bitwise and");
    assert((0xf0 | 0x0f) == 0xff, "Bitwise or");
    assert(1 << 4 == 16, "Left shift");
    assert(256 >> 3 == 32, "Right shift");
```

‘!’ flips a boolean from true to false or from false to true.

```
    assert(!false, "Not false is true");
    assert(!(1 > 2), "Not (false comparison) is true");
```

Like other values, booleans can be formatted inside a text using alignment specifiers. ‘^’ centres the value in a field of the given width.

```
    assert("1{true:^7}2" == "1 true  2", "Boolean alignment");
}
```

7. Float

A ‘float’ stores a number with a decimal point, like 3.14 or −0.001. It uses 64-bit precision (the same as f64 in Rust), which is enough for most scientific and financial calculations.

```
fn main() {
```

Any literal that contains a decimal point is a float.

```
    assert(1.5 + 0.5 == 2.0, "Float addition");
    assert(3.0 / 2.0 == 1.5, "Float division");
    assert(2.0 * 1.5 == 3.0, "Float multiplication");
```

Adding the ‘f’ suffix makes the literal a single-precision (32-bit) float. Use this when you need to match a specific 32-bit format or save memory.

```
    x = 0.1f + 2 * 1.0f;
    assert(x == 2.1f, "Single precision float");
```

You can convert between numbers and text using ‘as’ and format strings.

```
    assert(3 as float == 3.0, "Integer to float");
    assert("1.5" as float == 1.5, "Text to float");
    assert("{1.5}" == "1.5", "Float formatted as text");
```

7.0.1. Formatting Floats

The format specifier ‘{value:width.precision}’ controls how a float is displayed. ‘width’ sets the minimum total characters; ‘precision’ sets the decimal places. Omitting width leaves the total length unconstrained.

```
    assert("{1.2:4.2}" == "1.20", "Float with width and precision");
    assert("{334.1:.2}" == "334.10", "Float with precision only");
    assert("{1.4:5}" == "  1.4", "Float with width only");
```

7.0.2. Math Functions

‘PI’ is a built-in constant equal to approximately 3.14159.

```
    assert(round(PI * 1000.0) == 3142.0, "PI constant");
```

‘pow(base, exponent)’ raises a number to a power. ‘log(value, base)’ is the inverse: it tells you what exponent gives you that value.

```
    assert(log(pow(4.0, 5), 2) == 10.0, "log base 2 of 4^5");
```

‘sin’ and ‘cos’ expect their argument in radians, not degrees. sin(PI) is nearly zero; cos(PI) is exactly −1.

```
    assert(ceil(sin(PI) + cos(PI) * 1000) == -1000.0, "sin and cos");
```

‘abs()’ removes the sign, returning the distance from zero.

```
    assert(abs(-2.5) == 2.5, "Absolute value of float");
```

‘round()’ moves to the nearest whole number. ‘ceil()’ always moves up; ‘floor()’ always moves down. All three return a float, not an integer.

```
    assert(round(2.6) == 3.0, "round up");
    assert(round(2.4) == 2.0, "round down");
    assert(ceil(2.1) == 3.0, "ceil");
    assert(floor(2.9) == 2.0, "floor");
```

Single-precision floats can be used directly inside format strings.

```
    assert("a{0.1f + 2 * 1.0f}b" == "a2.1b", "Single-precision format");
}
```

8. Functions

Functions let you give a name to a reusable piece of logic. This page shows how to declare functions, provide default argument values, pass arguments by reference so a function can modify the caller's variable, and return early when needed.

8.0.1. Declaring Functions

Use 'fn' followed by the function name, a parameter list in parentheses, and an optional '-> type' for the return type. You do not need a 'return' keyword for the last expression — it is returned automatically.

```
fn greet(name: text, greeting: text = "Hello") -> text {
    greeting + ", " + name + "!"
}
```

8.0.2. Reference Parameters and Defaults

A function without a return type is called for its side effects. Prefix a parameter with '&' to pass it by reference. A reference parameter lets the function read and write the caller's variable directly. Default values are written as '= value' after the type and are used when the caller leaves that argument out.

```
fn add(a: &integer, b: integer, c: integer = 0) {
    a += b + c;
}
```

8.0.3. Early Return

Sometimes you want to stop a function before reaching the end. The 'return' keyword exits immediately and sends back the given value.

```
fn classify(n: integer) -> text {
    if n < 0 { return "negative"; }
    if n == 0 { return "zero"; }
    "positive"
}
```

8.0.4. Const and Reference Parameters

Mark a parameter 'const' when you want to guarantee that the function never modifies it. The compiler enforces this: any attempt to assign to a 'const' parameter is an error. Symmetrically, a '&' parameter promises the function will modify the value — declaring '&' when you never actually write to the parameter is also a compile error.

```
fn scale(a: const integer, factor: const integer) -> integer {
    a * factor
}
```

8.0.5. Type-Based Dispatch

```
fn describe_int(v: integer) -> text { "integer:{v}" }
fn describe_text(v: text) -> text { "text:{v}" }
```

```
fn main() {
```

Calling a function with all arguments provided explicitly.

```
    assert(greet("World", "Hi") == "Hi, World!", "Explicit argument");
```

Omitting the last argument lets the default value take effect.

```
    assert(greet("World") == "Hello, World!", "Default argument");
```

Because ‘a’ is a reference parameter, each call to ‘add’ changes the value of ‘v’.

```
    v = 1;
    add(v, 2);        // v = 1 + 2 = 3
    add(v, 4, 1);     // v = 3 + 4 + 1 = 8
    assert(v == 8, "Reference parameter: {v}");
```

‘classify’ uses early returns to handle each case separately.

```
    assert(classify(-5) == "negative", "Classify negative");
    assert(classify(0) == "zero", "Classify zero");
    assert(classify(3) == "positive", "Classify positive");
```

‘scale’ marks both parameters ‘const’, so the compiler verifies they are not changed.

```
    assert(scale(3, 7) == 21, "scale(3,7)");
```

Loft selects the right function based on the type of the argument.

```
    assert(describe_int(42) == "integer:42", "Integer describe");
    assert(describe_text("hi") == "text:hi", "Text describe");
}
```

9. Vector

A vector is a list of values that can grow and shrink at runtime. All elements must have the same type. Vectors are written with square brackets: '[1, 2, 3]'.

```
fn main() {
```

Create a vector with a literal. You can loop over it just like a range. '#first' is true for the very first element visited. '#index' gives the position of the current element (starts at 0).

```
    x = [1, 3, 6, 9];
    b = "";
    for v in x {
        if !v#first {
            b += " ";
        }
        b += "{v#index}:{v}"
    }
    assert(b == "0:1 1:3 2:6 3:9", "result {b}");
```

Use '+=' to append another vector to the end. You can remove elements while iterating by writing 'v#remove' inside the loop body. An 'if' clause after 'in vector' lets you visit only matching elements. Note: appending to a vector while iterating over it (v += [...] inside the loop) is a compile error — the iterator would visit newly added elements, risking an infinite loop.

```
    x += [12, 14, 15];
    for v in x if v % 3 != 0 {
        v#remove;
    }
    assert("{x}" == "[3,6,9,12,15]", "result {x}");
```

9.0.1. Slicing

'v[a..b]' gives you the elements from index a up to but not including b. Leaving out the start ('v[..b]') starts from 0. Leaving out the end ('v[a..]') goes to the last element.

```
    pows = [1, 2, 4, 8, 16];
    assert("{pows[1..3]}" == "[2,4]", "Sub-vector");
    assert("{pows[3..]}" == "[8,16]", "Open-ended sub-vector");
    assert("{pows[..3]}" == "[1,2,4]", "Open-start sub-vector");
```

Reading beyond the end of a vector returns null instead of crashing.

```
    assert(!pows[10], "Out-of-bounds access returns null");
```

9.0.2. Comprehensions

A comprehension builds a new vector from a loop in one expression. Write '[for variable in range { expression }]'. Add an 'if' clause to keep only certain elements.

```
    evens = [for n in 1..10 if n % 2 == 0 { n }];
    assert("{evens}" == "[2,4,6,8]", "Filtered comprehension");
    doubled = [for n in 1..6 { n * 2 }];
    assert("{doubled}" == "[2,4,6,8,10]", "Doubled comprehension");
```

A for loop placed directly inside a format string collects its results into a list. A format specifier after ':' is applied to every element.

```
    assert("{for n in 1..7 {n*2}:02}" == "[02,04,06,08,10,12]", "Formatted vector
loop");
```

9.0.3. Reverse Iteration

Wrap a range in 'rev()' to visit elements from last to first.

```
    c = 0;
    for e in pows[rev(0..=3)] {
        c = c * 10 + e;
    }
    assert(c == 8421, "Reverse sub-vector iteration");
```

A vector of structs can be created with a repeated element using '; count'. (See 08-struct.loft for struct syntax.)

Note: if you pass a vector to a function with '&' and append to it inside the function, the extra elements are not visible to the caller after the call returns. Only changes to existing elements (like setting a field) are reflected back. To return a larger vector, use a return value instead: fn with_extra(v: vector<integer>, x: integer) -> vector<integer> { v += [x]; v } items = with_extra(items, 99);

```
}
```

10. Structs

A struct groups related values under one name. Each piece of data is called a field. You access fields with dot notation: ‘value.field’.

```
struct Area {
    height: u16,
    terrain: u8,
    water: u8,
    direction: u8
}
```

10.0.1. Field Constraints

You can restrict what values a field may hold. ‘limit(min, max)’ rejects any value outside that range. ‘not null’ means zero is a real value — without it, zero would be treated as “no value”. Fields you leave out in a constructor get zero (or null for nullable fields) by default.

```
struct Point {
    r: integer limit(0, 255) not null,
    g: integer limit(0, 255) not null,
    b: integer limit(0, 255) not null
}
```

10.0.2. Methods

A method is just a function whose first parameter is named ‘self’. The type of ‘self’ determines which struct it belongs to. You call it with dot notation: ‘myPoint.value()’.

```
fn value(self: Point) -> integer {
    self.r * 0x10000 + self.g * 0x100 + self.b
}
```

10.0.3. Computed Fields

A field can calculate its value from other fields in the same struct. Write ‘= expression’ after the type to set a default that is evaluated at construction time. Inside that expression, ‘$’ refers to the struct being built.

```
struct Object {
    name_length: integer = len($.name),
    name: text
}
```

```
fn main() {
```

You can fill a vector with copies of the same struct using ‘; count’ syntax.

```
    map = [Area {height:0, terrain:1, water:1, direction:1}; 16 ];
    assert("{map[3]}" == "{{height:0,terrain:1,water:1,direction:1}}", "record
{map[3]}");
```

Fields left out of the constructor get their zero or null default.

```
    p = Point { r:128, b:128 };
    assert(p.g == 0, "Omitted field defaults to zero");
    assert("{p}" == "{{r:128,g:0,b:128}}", "Struct compact formatting");
```

10.0.4. Formatting Structs

The default format lists all fields compactly. ':j' produces JSON output, which is useful for storing or sending data.

```
    o = Object { name: "hello" };
    assert(o.name_length == 5, "Calculated field");
    assert("{o:j}" == "{{\"name_length\":5,\"name\":\"hello\"}}", "JSON format");
```

Call a method on a variable using dot notation. The return value can be formatted inline inside a text.

```
    purple = Point { r:128, b:128 };
    assert("{purple.value():x}" == "800080", "Method result in format");
```

You can call a method directly on a constructor expression without storing it first.

```
    assert(Point { r:255, g:0, b:0 }.value() == 0xff0000, "Method on
constructor");
```

Note: methods that return a new struct value (written as '-> MyStruct') do not yet work correctly at runtime. Methods that return a number, float, or text, and methods that modify 'self' in place, work fine.

```
}
```

defaults on fields & null defaults

11. Enums

An enum defines a fixed set of named choices. Plain enums are just names that can be compared and ordered. Struct enums go further: each variant can carry its own data.

Enum values can be compared with '==' and '<', ordered by declaration position, and converted to and from text.

```
enum Direction { North, East, South, West }
```

11.0.1. Struct Enums

Attach named fields to a variant by writing a struct body inside the enum. Each variant becomes its own type and can have its own methods.

```
enum Shape {
    Circle { radius: float },
    Rect { width: float, height: float }
}
```

11.0.2. Polymorphic Dispatch

Write the same function name for each variant. At the call site, Loft looks at the actual type of the value and calls the right version. This is called polymorphic dispatch — the behaviour changes depending on the variant.

```
fn area(self: Circle) -> float {
    PI * pow(self.radius, 2)
}
```

```
fn area(self: Rect) -> float {
    self.width * self.height
}
```

```
fn main() {
```

Plain enum values are ordered by the position they appear in the declaration. North comes first, so it is less than East, which is less than South, and so on.

```
    d = East;
    assert(d == East, "Enum equality");
    assert(d != North, "Enum inequality");
    assert(d > North, "East declared after North, so it is greater");
    assert(West > South, "West declared last, so it is greatest");
```

You can parse a text value into an enum using 'as EnumName'.

```
    assert("West" as Direction == West, "Text to enum");
```

Struct enum variants are constructed like regular structs. Use dot notation to read their fields.

```
    c = Circle { radius: 1.0 };
    assert(c.radius == 1.0, "Variant field access");
```

```
    r = Rect { width: 4.0, height: 5.0 };
    assert(r.width * r.height == 20.0, "Rect area manually");
```

Calling ‘area()’ on a Circle runs the Circle version; on a Rect it runs the Rect version. The correct function is chosen automatically at runtime based on the actual variant.

```
    assert(round(c.area() * 1000) == round(PI * 1000), "Circle area = PI*r^2");
    assert(r.area() == 20.0, "Rect area = width*height");
```

Formatting a plain enum value prints its variant name as text.

```
    assert("{d}" == "East", "Format plain enum");
```

If a variant intentionally has no implementation of a method, the compiler emits a warning. To silence the warning, provide an empty-body stub: fn area(self: SomeVariant) -> float { } A stub is callable at runtime (returns null) and suppresses the unused-parameter warning.

Note: polymorphic methods that build a text result using format strings with field access (like ‘fn describe(self: Circle) -> text { “r={self.radius}” }’) currently cause a runtime crash when defined on more than one variant of the same enum. A safe workaround is to return a number from the method and build the text in the caller.

```
}
```

12. Sorted

A ‘sorted’ collection holds records in order by one or more key fields. You can look up a record by key instantly and iterate all records in key order. The collection stays sorted as you add new elements — no manual sorting needed. Declare the key fields inside angle brackets: ‘field’ sorts ascending, ‘-field’ descending.

```
struct Elm { key: text, value: integer }
struct Db { map: sorted<Elm[-key]> }
```

```
fn main() {
```

12.0.1. Adding Elements

You initialise a sorted collection the same way as a vector. The elements are automatically placed in the right position as you add them.

```
    db = Db { map: [
        Elm { key: "One",   value: 1 },
        Elm { key: "Two",   value: 2 },
        Elm { key: "Three", value: 3 },
        Elm { key: "Four",  value: 4 }
    ]};
```

12.0.2. Looking Up by Key

Use square brackets with the key value to find a record instantly. If the key is not present the result is null — you can check it with ‘!’.

```
    assert(db.map["Two"].value == 2, "Key lookup: Two");
    assert(db.map["Three"].value == 3, "Key lookup: Three");
    assert(!db.map["Five"], "Missing key returns null");
    assert(!db.map[null], "Null key returns null");
```

Append new elements with ‘+=’; they are placed in the correct sorted position.

```
    db.map += [Elm { key: "Zero", value: 0 }];
    assert(db.map["Zero"].value == 0, "Newly added element");
```

12.0.3. Iterating in Order

A ‘for’ loop over a sorted collection visits elements in key order. Here the key is ‘-key’ (descending text), so the order is: Zero, Two, Three, One, Four.

```
    sum = 0;
    for v in db.map {
        sum = sum * 10 + v.value;
    }
```

Zero(0), Two(2), Three(3), One(1), Four(4) ⟶ 0*10+2=2, *10+3=23, *10+1=231, *10+4=2314

```
assert(sum == 2314, "Sorted iteration total: {sum}");
```

Note: iterating a sorted collection in reverse (for v in rev(db.map)) is not yet supported. To go the other way, collect into a plain vector first, then reverse that.

12.0.4. Loop Helpers: #first, #count, and #remove

'v#first' is true for the very first element visited. 'v#count' is a running index starting at 0. 'v#remove' removes the current element while iterating.

```
first_key = "";
labels = "";
for v in db.map {
    if v#first { first_key = v.key }
    if !v#first { labels += "," }
    labels += "{v#count}:{v.key}"
}
assert(first_key == "Zero", "First element: {first_key}");
assert(labels == "0:Zero,1:Two,2:Three,3:One,4:Four", "Labels: {labels}");
```

Remove elements with value > 2 while iterating.

```
for v in db.map if v.value > 2 {
    v#remove
}
sum2 = 0;
for v in db.map { sum2 += v.value }
```

Remaining: Zero(0), Two(2), One(1) ⟶ sum = 3

```
assert(sum2 == 3, "Sum after remove: {sum2}");
```

12.0.5. Removing by Key

Assigning null to a sorted subscript removes the element with that key. Removing a key that is not present is a safe no-op.

```
db2 = Db { map: [
    Elm { key: "A", value: 10 },
    Elm { key: "B", value: 20 },
    Elm { key: "C", value: 30 }
]};
db2.map["B"] = null;
assert(!db2.map["B"],          "B was removed");
assert(db2.map["A"].value == 10, "A still present");
assert(db2.map["C"].value == 30, "C still present");
db2.map["missing"] = null;   // no-op; does not panic
sum3 = 0;
for v in db2.map { sum3 += v.value }
assert(sum3 == 40, "Sum after key removal: {sum3}");
```

Note: #index is not available on sorted — use #count for a sequential counter.

```
}
```

filling and finding values

13. Index

An 'index' lets you find records instantly by key and iterate over ranges of keys in order. It supports multi-part keys: you can sort by a primary key and break ties with a secondary key. Declare the key fields inside angle brackets: 'field' sorts ascending, '-field' descending. Note: each record can only belong to one index at a time.

```
struct Elm { nr: integer, key: text, value: integer }
struct Db { map: index<Elm[nr, -key]> }
```

```
fn main() {
```

13.0.1. Adding and Looking Up Records

Fill the index just like a vector. Elements are automatically kept in key order. This index is sorted first by 'nr' (ascending), then by 'key' (descending) for ties.

```
    db = Db { map: [
        Elm { nr: 101, key: "One",   value: 1 },
        Elm { nr: 92,  key: "Two",   value: 2 },
        Elm { nr: 83,  key: "Three", value: 3 },
        Elm { nr: 83,  key: "Four",  value: 4 },
        Elm { nr: 83,  key: "Five",  value: 5 },
        Elm { nr: 63,  key: "Six",   value: 6 },
    ]};
```

Provide all key fields together inside brackets to find a record. Supply fewer fields to match all records that share a prefix.

```
    assert(db.map[101, "One"].value == 1, "Key lookup");
    assert(!db.map[12, ""], "Missing key returns null");
    assert(!db.map[83, "One"], "Wrong secondary key returns null");
```

13.0.2. Iterating in Order

A 'for' loop visits all elements in key order.

```
    total = 0;
    for r in db.map {
        total += r.value;
    }
    assert(total == 21, "Sum of all values: {total}");
```

13.0.3. Range Queries

Provide a range in the first key position to visit only the matching slice. '[83..92, "Two"]' means: nr from 83 up to (not including) 92, and key from "Two" down (since the key is sorted descending, "Two" is the start of that descending segment).

```
    sum = 0;
    for v in db.map[83..92, "Two"] {
        sum = sum * 10 + v.value;
    }
```

Elements matched: (83, Three, 3), (83, Four, 4), (83, Five, 5)

```
    assert(sum == 345, "Range iteration result: {sum}");
```

13.0.4. Loop Helpers: #first and #count

'r#first' is true for the very first element visited. 'r#count' is a running counter starting at 0. Note: #index is not meaningful on index collections — use #count instead.

```
    first_nr = 0;
    count_total = 0;
    for r in db.map {
        if r#first { first_nr = r.nr }
        count_total = r#count
    }
    assert(first_nr == 63, "First element by key: {first_nr}");
    assert(count_total == 5, "Total elements: {count_total}");
```

13.0.5. Removing by Key

Assigning null to an index subscript removes the element with that exact key combination. Removing a key that is not present is a safe no-op.

```
    db.map[92, "Two"] = null;
    assert(!db.map[92, "Two"],       "element (92, Two) was removed");
    assert(db.map[101, "One"].value == 1, "element (101, One) still present");
    db.map[999, "Z"] = null;   // no-op; does not panic
    total2 = 0;
    for r in db.map { total2 += r.value }
    assert(total2 == 19, "Sum after key removal: {total2}");
}
```

14. Hash

A 'hash' lets you find a record by its key in constant time, regardless of how many records you have. Unlike 'sorted' and 'index', a hash has no meaningful order — you use it purely for fast lookups. List the key fields in brackets: 'hash<Type[field]>'. Combine a hash with a vector when you want both fast lookup and a stable iteration order.

```
struct Keyword { name: text }
struct Data { h: hash<Keyword[name]> }
```

14.0.1. Combining Hash and Vector

Pairing a hash with a vector on the same record type gives you the best of both worlds: the hash for instant lookup by key, and the vector for iterating in insertion order.

```
struct Count { t: text, v: integer }
struct Counting {
    entries: vector<Count>,
    lookup:  hash<Count[t]>
}
```

```
fn fill(c: Counting) {
    c.entries = [
        { t: "One",   v: 1 },
        { t: "Two",   v: 2 },
        { t: "Three", v: 3 },
        { t: "Four",  v: 4 }
    ]
}
```

```
fn main() {
```

14.0.2. Basic Lookup

Fill a hash the same way you fill a vector. Look up by key with square brackets. A found record is truthy; a missing key returns null.

```
    d = Data {};
    d.h = [{ name: "one" }, { name: "two" }];
    d.h += [{ name: "three" }, { name: "four" }];
```

```
    assert(d.h["three"], "Key exists");
    assert(!d.h["None"], "Missing key returns null");
```

14.0.3. Hash + Vector Together

Here 'entries' is the vector (keeps insertion order) and 'lookup' is the hash (fast access). Both fields point to the same records — adding to one automatically updates the other.

```
    c = Counting {};
    fill(c);
    assert(c.lookup["Three"].v == 3, "Hash lookup: Three");
    assert(c.lookup["One"].v == 1, "Hash lookup: One");
    assert(!c.lookup["Five"], "Missing key returns null");
```

Iterate in insertion order via the vector; look up by name via the hash. The vector also supports #first, #count, and #remove inside the loop.

```
    add = 0;
    for item in c.entries {
        add += item.v;
    }
    assert(add == 10, "Sum via vector iteration: {add}");
```

14.0.4. Removing a Key

Assigning null to a hash subscript removes that element. Removing a key that is not present is a safe no-op.

```
    d.h["three"] = null;
    assert(!d.h["three"], "three was removed");
    assert(d.h["one"],    "one still present");
    d.h["missing"] = null;   // no-op; does not panic
```

14.0.5. Why you cannot iterate a hash directly

A hash has no stable element order — hash bucket positions depend on key hashes and the internal load factor, so there is no meaningful #index or sequential position. ‘for item in c.lookup’ is therefore a compile error. Always pair a hash with a vector when you need both fast lookup and ordered iteration.

```
}
```

15. File

A file handle lets you read and write files without worrying about when the OS opens or closes them. The file opens on the first read or write and closes automatically when the handle goes out of scope.

15.0.1. Inspecting the File System

`file(path)` creates a File handle without opening anything yet. `f\#format` tells you what kind of path you are looking at: TextFile, LittleEndian, BigEndian, Directory, or NotExists. `lines()` reads a text file and returns it as a vector of lines. `exists(path)` is a convenience shorthand for checking that a path is not NotExists. `delete(path)` removes a file and returns false if it was not there.

Clean up any leftover files from a previous interrupted run.

```
fn cleanup() {
    delete("test.bin");
    delete("test2.bin");
    delete("buffer.bin");
}
```

```
struct Buffer {
   size: i32,
   data: vector<single>
}
```

```
fn main() {
    cleanup();
```

Asking for the format of a directory path returns Directory, not a file format. `lines()` reads the named text file and splits it on newlines.

```
    ex = file("example");
    assert(ex#format == Directory, "example is a directory");
    c = file("example/config/terrain.txt").lines();
    assert(c[1] == "    terrain = [", "Line was '{c[1]}'");
```

`exists` and `delete` are safe to call when the file is absent. `delete` returns false rather than crashing when nothing is there.

```
    assert(!exists("test.bin"), "File should not exist before the test.");
    assert(!delete("nonexistent_xyz.bin"), "delete on missing file returns
false");
```

15.0.2. Writing a Text or Binary File

Wrapping the file handle in a block ensures the file closes the moment the block ends. Set `f\#format` to LittleEndian or BigEndian before writing binary data. Use `f += value` to append the raw bytes of any scalar value (u8, u16, i32, long, single, float, or text).

```
    { f = file("test.bin");
      assert(f#format == NotExists, "File should not exist yet.");
      f#format = BigEndian;
      f += 0 as u8;
      f += 1 as u8;
      f += 0x203 as u16;
      f += 0x4050607;
      f += 0x8090a0b0c0d0e0fl;
```

`f\#size` returns the total number of bytes written so far.

```
      assert(f#size == 16l, "Should have written 16 bytes.");
```

Text is written as raw UTF-8 bytes with no length prefix.

```
      f += "Hello world!";
      assert(f#size == 28l, "Size should be 28 bytes (16 + 12).");
    } // The file closes when the handle goes out of scope.
```

`move(src, dst)` renames a file. It refuses if the destination already exists or if either path would leave the project directory.

```
    assert(exists("test.bin"), "File should exist after writing.");
    assert(!move("test.bin", "../test.bin"), "Should refuse to move outside the
project.");
    assert(move("test.bin", "test2.bin"), "Could not move the file.");
```

15.0.3. Reading Back What You Wrote

When you open an existing file the default format is TextFile. Set `f\#format` to match the format used when writing before you read any bytes. `f\#read(n) as T` reads exactly n bytes and interprets them as type T. `f\#index` is the byte offset where the last read started. `f\#next` is the current read position; assign to it to seek anywhere.

```
    { f = file("test2.bin");
      assert(f#format == TextFile, "The default format is TextFile.");
      f#format = LittleEndian;
```

The file was written as BigEndian, so reading the same 4 bytes as LittleEndian produces a byte-swapped value — that is intentional here and confirms that the bytes were stored in the order you chose.

```
      v = f#read(4) as i32;
      assert(v == 0x3020100, "BigEndian bytes 0..3 read as LittleEndian i32.");
      assert(f#index == 0l, "Last read started at byte 0.");
      assert(f#next == 4l, "Next read starts at byte 4.");
```

Seek to byte 16 to skip past the integers and read the text directly.

```
      f#next = 16l;
      s = f#read(5) as text;
      assert(s == "Hello", "Partial text read from offset 16.");
      assert(f#index == 16l, "Last read started at byte 16.");
      assert(f#next == 21l, "Next position after 5-byte text read.");
```

Requesting more bytes than remain simply returns whatever is left.

```
      rest = f#read(100) as text;
      assert(rest == " world!", "Read continues to end of file.");
      assert(f#next == f#size, "Position should be at end of file.");
```

You can seek back to any position and re-read.

```
      f#next = 0l;
      assert(f#read(4) as i32 == 0x3020100, "Seek-and-reread matches original.");
    }
    assert(delete("test2.bin"), "Could not remove the test file.");
```

15.0.4. Working with Vectors and Struct Data

`f += vector\<T\>` writes every element in sequence as raw bytes, which is the fastest way to dump a whole collection to disk. Setting `f\#size = n` truncates or zero-extends the file to exactly n bytes — useful for discarding the tail after you have finished writing.

```
    { f = file("buffer.bin");
      f#format = LittleEndian;
      ints = [1, 2, 3, 4];
      f += ints;
      assert(f#size == 16l, "Four i32 values = 16 bytes");
```

Truncate to the first two integers.

```
      f#size = 8l;
      assert(f#size == 8l, "Truncated to 8 bytes");
    }
    assert(delete("buffer.bin"), "Could not remove buffer.bin after vector write
test.");
```

Write a count followed by individual float values. `sizeof(T)` returns the byte width of a type, so you can compute the correct read length without hard-coding magic numbers.

```
    { buf = file("buffer.bin");
      buf#format = LittleEndian;
      buf += 4 as i32;
      buf += 1.1f;
      buf += 1.2f;
      buf += 2.1f;
```

```
      buf += 2.2f;
    }
```

Read the count, then use it to read exactly that many floats directly into a struct field. This pattern lets you serialise and deserialise structs with variable-length data cleanly.

```
    { b = Buffer {};
      f = file("buffer.bin");
      f#format = LittleEndian;
      n = f#read(4) as i32;
      b.data = f#read(n * sizeof(single));
    }
    assert(delete("buffer.bin"), "Could not remove buffer.bin.");
}
```

16. Image

Loft has a built-in Image type for loading PNG files and inspecting their pixels. This page has no runnable examples because PNG test files are not included in the standard test suite, but the API shape below shows you exactly how to use it in your own code.

16.0.1. Loading an Image

Pass a File handle to the `png()` function to load a PNG into memory. The file is read in full and decoded; after that you can access pixels without any further I/O.

img = png(file("photo.png"))

16.0.2. Reading Dimensions

Once loaded, `img\#width` and `img\#height` give you the pixel dimensions.

w = img#width h = img#height println("image is {w} x {h} pixels")

16.0.3. Accessing Individual Pixels

Index the image with two coordinates to get a pixel value. Each pixel carries four channels: red, green, blue, and alpha, all in the range 0–255.

px = img[x, y] r = px#red g = px#green b = px#blue a = px#alpha

16.0.4. Iterating Over All Pixels

A `for` loop over an image visits every pixel from top-left to bottom-right. This is the easiest way to scan or transform the whole image.

for px in img { if px#red > 200 { println("bright red pixel") } }

```
fn main() {
}
```

17. Lexer

The lexer library breaks a text into tokens so your program can understand its structure. Tokens are the smallest meaningful pieces: numbers, identifiers, operators, and string literals. You tell the lexer which multi-character sequences count as single tokens and which words are reserved, and it handles the rest.

17.0.1. Setting Up the Lexer

Create a `lexer::Lexer` and register your language's rules before you parse anything. `set_tokens` ensures operators like `+=` or `\>\>` are scanned as one token instead of two separate characters. `set_keywords` prevents reserved words from being returned as plain identifiers — the lexer will report them exactly as written so your parser can treat them specially.

```
use lexer;
```

```
fn main() {
    l = lexer::Lexer {};
    l.set_tokens(["+=", "*=", "-=", "<=", ">=", "!=", "==", ">>", "<<", "->",
"=>", ">>>", "..", "..=", "&&", "||"]);
    l.set_keywords(["for", "in", "if", "else", "fn", "pub", "use", "struct",
"enum", "match", "and", "or"]);
```

17.0.2. Reading Tokens

`parse_string(name, source)` feeds source text into the lexer. The name is used in error messages and position reports. After that, call the typed reader functions one by one to consume tokens in order.

`int()` consumes and returns the next integer token, or null if the current token is not an integer. `long_int()` does the same for integers suffixed with `l`. `matches(s)` consumes the next token only when it equals s and returns true; otherwise it leaves the token in place and returns false. `peek()` returns the next token as text without consuming it. `position()` returns the current location as `file:line:col`.

```
    l.parse_string("Tokens", "12 += -2 * 3l >> 4");
    assert(l.int() == 12, "Integer");
    assert(!l.matches("+"), "Incorrect plus");
    assert(l.peek() != "+", "Incorrect plus");
    assert(l.matches("+="), "Incorrect plus_is");
    assert(l.int() == -2, "Second integer");
    assert(l.matches("*"), "Incorrect multiply");
    assert(l.int() != 3, "Third number");
    assert(l.long_int() == 3, "Incorrect long");
    assert(l.position() == "Tokens:1:15", "Incorrect position {l.position()}");
    assert(!l.matches(">"), "Incorrect higher");
    assert(l.matches(">>"), "Incorrect logical shift");
    assert(l.position() == "Tokens:1:18", "Incorrect position {l.position()}");
```

17.0.3. String Literals and Comments

`constant_text()` reads a double-quoted string literal and unescapes any escape sequences inside it. `constant_character()` reads a single-quoted character literal and returns it as text.

```
    l.parse_string("Texts", "\"123\" + '4'");
    assert(l.constant_text() == "123", "Incorrect text literal");
    assert(l.matches("+"), "Incorrect add");
    assert(l.constant_character() == "123", "Incorrect text literal");
```

The lexer collects `//` comments automatically as it scans. You do not need to handle them yourself. `last_comment()` returns the accumulated comment text since the last consumed token. When multiple comment lines appear in a row they are joined with newlines into a single string. `comment_behind()` is true when the comment appeared on the same line as the preceding token rather than on its own line above. `is_finished()` returns true once every token has been consumed.

```
    l.parse_string("Comments", "// starting comments\n123 // same line
comment\n// extra comment\n4");
    assert(!l.comment_behind(), "Initial comment not behind");
    assert(l.last_comment() == "starting comments", "Initial comment");
    assert(l.int() == 123, "Content integer");
    assert(l.comment_behind(), "Second comment is behind");
    assert(l.last_comment() == "same line comment\nextra comment", "Second
comment");
    assert(!l.is_finished(), "Not Ready");
    assert(l.int() == 4, "Second integer");
    assert(l.last_comment() == "", "No remaining comment");
    assert(l.is_finished(), "Ready");
```

17.0.4. Embedded Format Expressions

Loft string literals can embed expressions with `{expr}`. The lexer exposes a protocol that lets you parse these yourself. When `constant_text()` reaches a `{`, it returns the literal text before it and sets `is_formatting()` to true. At that point call `set_formatting(false)` and parse the embedded expression normally using the usual token readers. When the expression is done, call `set_formatting(true)` and consume the closing `}}`. Then `constant_text()` continues with the next segment of the string.

```
    l.parse_string("Formatting", "\"abc{{12 + 34}}def\"");
    assert(l.constant_text() == "abc", "Before formatting");
    assert(l.is_formatting(), "Formatting");
    l.set_formatting(false);
    assert(l.int() == 12, "First integer");
    assert(l.matches("+"), "Incorrect plus");
    assert(l.int() == 34, "Second integer");
    l.set_formatting(true);
    assert(l.matches("}}"), "Incorrect closing brace");
    assert(l.constant_text() == "def", "After formatting");
    assert(!l.is_formatting(), "Formatting");
}
```

18. Parser

Parser library: parsing loft source code programmatically from within a loft program.

`parse(name, source)` is the single entry point. It takes a display name (used in error messages) and a loft source string. The name argument does not need to correspond to a real file; it is only used when reporting parse errors.

The parser understands the full loft grammar:

- `struct Name { field: type \[= default\] }` — named-field aggregates
- `enum Name { Variant \[{ field: type }\] }` — tagged unions with optional payload
- `fn name(params) \[-\> type\] { body }` — functions with a block body
- `fn name(params) \[-\> type\]; \#rust "template"` — operator templates backed by Rust
- `use module;` — module imports
- Expressions: binary operators with precedence, function calls, field access,

index expressions, if/else, for loops, blocks, and formatted string literals

- Type expressions: plain names, generic types like `vector\<T\>`, sorted/hash/index

collections with key fields, and integer ranges with `limit(min, max)`

```
use parser;
```

```
fn main() {
```

Parse a minimal function definition. The double braces `{{` and `}}` are the loft way to write literal braces inside a format string.

```
    parser::parse("hello", "fn main() {{ println(\"Hello World\"); }}");
```

Parse a struct definition followed by a function that references it. The parser validates the syntax but does not execute the code.

```
    parser::parse("point", "struct Point {{ x: integer, y: integer }} fn origin()
-> Point {{ Point {{ x: 0, y: 0 }} }}");
```

Parse an enum with a plain variant and a variant carrying fields.

```
    parser::parse("shape", "enum Shape {{ Circle {{ radius: float }}, Rectangle
{{ w: float, h: float }} }}");
}
```

19. Libraries

A library is a `.loft` file you can share across projects. Place it in a `lib/` directory, import it with `use name;` at the top of your file, and then refer to its types and functions using the `name::` prefix. Everything in the library is always accessible — there is no private/public barrier.

`use` statements must come before any `fn` or `struct` definition in your file. Putting a `use` after a definition is a syntax error.

```
use testlib;
```

19.0.1. Constants and Enums

Library constants and enum variants are accessed with the `libname::` prefix. You can compare, order, and convert enum values exactly as you would with enums defined in the same file.

19.0.2. Struct Construction and Field Access

Construct a library struct with its full namespace prefix. Omitting the prefix is a parse error. Once you have a value, field access uses the plain dot notation with no prefix needed.

19.0.3. Calling Library Methods

Methods defined in the library are called on the value directly — no prefix on the call site. Free functions (not methods) do need the `libname::` prefix.

19.0.4. Extending a Library Type

You can add new methods to a library type from your own file. Write the `self` parameter type with the `::` separator and the compiler treats the function as a method on that type.

```
fn shifted(self: testlib::Point, dx: float, dy: float) -> testlib::Point {
    testlib::Point { x: self.x + dx, y: self.y + dy }
}
```

```
fn main() {
```

Constants from a library require the library prefix.

```
    assert(testlib::MAX_SIZE == 100, "library constant MAX_SIZE");
    assert(testlib::MIN_SIZE == 1, "library constant MIN_SIZE");
```

Library enum variants: comparison and ordering work as normal.

```
    s = testlib::Ok;
    assert(s == testlib::Ok, "enum equality");
    assert(s < testlib::Error, "enum ordering");
    assert("{s}" == "Ok", "enum to text: {s}");
    assert("Warning" as testlib::Status == testlib::Warning, "text to enum");
```

Library enum values can be passed to library free functions.

```
    assert(testlib::status_text(testlib::Error) == "err", "free fn with enum
arg");
```

Structs are constructed with namespace prefix and accessed by field.

```
    p = testlib::Point { x: 3.0, y: 4.0 };
    assert(p.x == 3.0, "field access: x={p.x}");
    assert(p.y == 4.0, "field access: y={p.y}");
```

Methods defined in the library are called without a prefix.

```
    dist = p.distance();
    assert(round(dist) == 5.0, "method call: distance={dist}");
```

User-defined method on a library type (self: testlib::Point) works.

```
    assert(p.shifted(1.0, 2.0).x == 4.0, "shifted x");
    assert(p.shifted(0.0, 2.0).y == 6.0, "shifted y");
```

Scalar fields can be mutated directly.

```
    b = testlib::Bag { label: "test", count: 0 };
    b.label = "updated";
    assert(b.label == "updated", "direct scalar field mutation");
```

Methods that mutate scalar fields work.

```
    b.bump();
    assert(b.count == 1, "method scalar mutation: count={b.count}");
    b.bump();
    assert(b.count == 2, "method scalar mutation: count={b.count}");
```

Free functions: called with the library prefix.

```
    assert(testlib::add(3, 4) == 7, "free function add");
    assert(testlib::add(0, -5) == -5, "free function negative");
```

Library types work as function parameter types when written with their full namespace prefix in the function signature.

A struct with a vector field can be initialised via a struct literal, including elements that are themselves library structs.

```
    bag2 = testlib::Bag { label: "full", count: 3, items: [
        testlib::Point { x: 1.0, y: 0.0 },
        testlib::Point { x: 0.0, y: 1.0 }
    ]};
```

```
    assert(len(bag2.items) == 2, "initialized vector field: {len(bag2.items)}");
    assert(bag2.items[0].x == 1.0, "vector field element access");
```

Importing the same library twice is silently ignored. A library can itself import other libraries using `use`, so dependency chains work.

19.0.5. Limitations

These are the current rough edges to keep in mind.

`use` must appear before all definitions. If you write a function first and then a `use`, the compiler reports a syntax error: fn foo() {} use testlib;

All names from a library must be written with the `name::` prefix. Writing `Point {}`, `MAX_SIZE`, or `add(1, 2)` without the prefix causes a parse error or an unknown-name error.

`pub` on struct fields is not supported and causes a parse error. Writing `pub` on a top-level `struct` or `fn` is accepted but has no effect — all library definitions are always visible to importers.

Appending to a scalar vector field with `+=` on an existing variable does not persist. Initialise vector fields via the struct literal instead: b.items += p

```
}
```

20. Store Locks

A store lock prevents accidental modification of a value after it has been set up. Loft automatically locks stores that are passed as `const` parameters so the compiler can catch any unintended writes at compile time. You can also set the lock flag by hand when you want to protect a value in your own code.

```
struct Counter {
    value: integer
}
```

A `const` parameter signals that this function will not modify the store. In debug builds the runtime locks the store automatically when you call this function.

```
fn read_value(self: const Counter) -> integer {
    self.value
}
```

A non-const parameter leaves the store unlocked so the function can write to it freely.

```
fn increment(self: Counter) {
    self.value += 1
}
```

```
fn main() {
```

20.0.1. Checking and Setting a Lock

A freshly created reference starts unlocked.

```
    c = Counter { value: 10 };
    assert(!c#lock, "New store starts unlocked");
```

Locking the store makes the flag true.

```
    c#lock = true;
    assert(c#lock, "Store is locked after d#lock = true");
```

20.0.2. The const Keyword for Local Variables

Declaring a local variable with `const` signals your intent that the value will not change. In debug builds the runtime locks the store immediately after initialisation so any accidental write is caught. In release builds the lock is not set automatically — only your explicit `\#lock = true` has an effect.

```
    const d = Counter { value: 42 };
    assert(read_value(d) == 42, "const local: value readable");
```

The lock state may differ between debug and release builds, but the stored value is always correct either way.

```
    lock_state = d#lock;
    assert(lock_state == true || lock_state == false, "d#lock is a boolean");
```

20.0.3. Passing const to a Function

When you pass a value to a `const` parameter, the function can read it but not write to it. In debug builds the store is locked for the duration of the call and unlocked when the call returns.

```
    result = read_value(c);
    assert(result == 10, "const arg: value readable after lock");
```

A non-const reference starts unlocked and you can lock it yourself.

```
    e = Counter { value: 5 };
    assert(!e#lock, "non-const starts unlocked");
    e#lock = true;
    assert(e#lock, "non-const lock set to true");
```

Reading the lock state before and after setting it lets you verify the transition explicitly.

```
    f = Counter { value: 7 };
    before = f#lock;
    f#lock = true;
    after = f#lock;
    assert(!before, "lock was false before setting");
    assert(after, "lock is true after setting");
```

`get_store_lock(ref)` is the function form of the `\#lock` property. Both return the same boolean, so you can use whichever reads more naturally in context.

```
    g = Counter { value: 99 };
    assert(get_store_lock(g) == g#lock, "get_store_lock matches d#lock read");
    g#lock = true;
    assert(get_store_lock(g) == g#lock, "get_store_lock matches after lock");
}
```

21. Parallel execution

The `par(b=worker_call, threads)` clause on a `for` loop runs a function on every element of a vector in parallel and gives you the results one by one in the loop body. Use it when you have a large collection and a CPU-intensive per-element calculation: the work is spread across the requested number of threads and the results come back in the original order.

The full syntax is: for a in <vector> par(b=<worker_call>, <threads>) { body }

Two worker call forms are supported. Form 1 calls a global or user-defined function with the loop element as its argument: for a in items par(b=my_func(a), 4) { … }

Form 2 calls a method on the element itself: for a in items par(b=a.my_method(), 4) { … }

The worker function must take a `const` reference to the element type and return a single primitive value (integer, float, or boolean).

—— Shared struct definitions ——

```
struct Score { value: integer }
struct ScoreList { items: vector<Score> }
```

```
struct Range { lo: integer, hi: integer }
struct RangeList { items: vector<Range> }
```

—— Worker functions ——

Global functions (Form 1)

```
fn double_score(r: const Score) -> integer { r.value * 2 }
fn span(r: const Range) -> integer { r.hi - r.lo }
fn score_as_float(r: const Score) -> float { r.value as float }
fn score_positive(r: const Score) -> boolean { r.value > 0 }
```

Methods on Score (Form 2)

```
fn get_value(self: const Score) -> integer { self.value }
fn is_positive(self: const Score) -> boolean { self.value > 0 }
```

—— Helpers ——

```
fn make_scores() -> ScoreList {
    q = ScoreList {};
    q.items += [Score{value:10}, Score{value:20}, Score{value:30}];
    q
}
```

```
fn make_ranges() -> RangeList {
    q = RangeList {};
    q.items += [Range{lo:0, hi:10}, Range{lo:5, hi:12}, Range{lo:-3, hi:7}];
    q
}
```

```
fn main() {
```

21.0.1. Running a Global Function in Parallel

Each Score's value is doubled by `double_score`. With 1 thread the work is sequential; with 4 threads it runs concurrently. Both must produce the same total because results are delivered in the original order.

```
    q = make_scores();
    sum = 0;
    for a in q.items par(b=double_score(a), 1) {
        sum += b;
    }
    assert(sum == 120, "form-1 integer (1 thread): sum == 120");
```

integer return, 4 threads

```
    q2 = make_scores();
    sum2 = 0;
    for a in q2.items par(b=double_score(a), 4) {
        sum2 += b;
    }
    assert(sum2 == 120, "form-1 integer (4 threads): sum == 120");
```

`span` works on a two-field struct. The element size is inferred from the struct layout so you do not need to specify it.

```
    r = make_ranges();
    span_sum = 0;
    for a in r.items par(b=span(a), 2) {
        span_sum += b;
    }
    assert(span_sum == 27, "form-1 Range span: sum == 27");
```

Workers can return float. Here we just count iterations to confirm every element was processed.

```
    q3 = make_scores();
    fcount = 0;
    for a in q3.items par(b=score_as_float(a), 1) {
        fcount += 1;
    }
    assert(fcount == 3, "form-1 float return: 3 elements processed");
```

Workers can return boolean. Use `if b` in the body to act on the result.

```
    q4 = make_scores();
    pos = 0;
    for a in q4.items par(b=score_positive(a), 1) {
        if b { pos += 1; }
    }
    assert(pos == 3, "form-1 boolean: all 3 positive");
```

An empty vector is safe: the loop body simply never executes.

```
    empty = ScoreList {};
    empty_sum = 0;
    for a in empty.items par(b=double_score(a), 1) {
        empty_sum += b;
    }
    assert(empty_sum == 0, "form-1 empty: body executes 0 times");
```

21.0.2. Running a Method in Parallel

Form 2 calls the worker as a method on each element. The syntax `b=a.get_value()` tells the compiler to dispatch `get_value` on every element in parallel and bind each result to `b` in the loop body.

```
    q5 = make_scores();
    sum3 = 0;
    for a in q5.items par(b=a.get_value(), 1) {
        sum3 += b;
    }
    assert(sum3 == 60, "form-2 integer (1 thread): sum == 60");
```

integer return, 4 threads

```
    q6 = make_scores();
    sum4 = 0;
    for a in q6.items par(b=a.get_value(), 4) {
        sum4 += b;
    }
    assert(sum4 == 60, "form-2 integer (4 threads): sum == 60");
```

boolean return — count positives via method

```
    q7 = make_scores();
    pos2 = 0;
    for a in q7.items par(b=a.is_positive(), 1) {
        if b { pos2 += 1; }
    }
    assert(pos2 == 3, "form-2 boolean: all 3 positive");
}
```

22. Logging

Logging lets you record events at runtime without stopping the program. Instead of printing everything to the console, you write to a log so you can filter by severity, redirect output to a file, or silence it entirely in production — all without changing your source code.

There are four severity levels. Use `log_info` for routine events you want to trace during development. Use `log_warn` when something unexpected happened but the program can continue safely. Use `log_error` when something went wrong and you need to investigate. Use `log_fatal` for the most serious problems that indicate a condition the program may not be able to recover from.

To configure where log output goes, place a `log.conf` file next to your `.loft` file or pass `--log-conf path/to/log.conf` on the command line. When no logger is configured the log calls are silent no-ops, which is why the tests below produce no visible output.

`assert(condition, message)` is related to logging in production mode: when assertions are disabled (release builds without the debug flag) a failing assert logs an error rather than aborting the program, so your service stays alive and the problem is captured in the log.

```
fn main() {
```

Call all four severity levels — no logger configured, so these are no-ops

```
    log_info("test info message");
    log_warn("test warn message");
    log_error("test error message");
    log_fatal("test fatal message");
```

assert with a true condition must not abort

```
    assert(true, "this should not fail");
```

```
    println("logging test passed");
}
```

23. Standard Library

23.1. Types

Primitive types built into lav.

```
pub type boolean size(1)
```

True or false value.

```
pub type integer size(4)
```

32-bit signed integer.

```
pub type long size(8)
```

64-bit signed integer. Use when values exceed 2 billion.

```
pub type single size(4)
```

32-bit floating-point. Good for graphics and performance-sensitive math.

```
pub type float size(8)
```

64-bit floating-point. Use when precision matters.

```
pub type text size(4)
```

UTF-8 string.

```
pub type character size(4)
```

A single Unicode code point.

```
pub type u8=integer limit(0, 255) size(1)
```

Integer subtypes for compact storage in struct fields. Behave as integer in expressions. 0 – 255, 1 byte.

```
pub type i8=integer limit(-128, 127) size(1)
```

–128 – 127, 1 byte.

```
pub type u16=integer limit(0, 65535) size(2)
```

0 – 65535, 2 bytes.

```
pub type i16=integer limit(-32768, 32767) size(2)
```

−32768 – 32767, 2 bytes.

```
pub type i32=integer size(4)
```

Full 32-bit integer range, 4 bytes.

23.2. Math

Functions for numeric computation. All trigonometric functions work in radians. Both single and float variants exist for every function — choose single for speed, float for precision.

```
pub fn abs(both: integer) -> integer
```

Absolute value. Removes the sign from a negative integer.

```
pub fn abs(both: long) -> long
```

Absolute value for long integers.

```
pub fn abs(both: single) -> single
```

Absolute value for single-precision floats.

```
pub fn cos(both: single) -> single
```

Cosine. Use for circular motion: x = r * cos(angle).

```
pub fn sin(both: single) -> single
```

Sine. Use for circular motion: y = r * sin(angle).

```
pub fn tan(both: single) -> single
```

Tangent. Use for slopes and perspective projection.

```
pub fn acos(both: single) -> single
```

Arc cosine. Returns the angle (in radians) whose cosine is v.

```
pub fn asin(both: single) -> single
```

Arc sine. Returns the angle whose sine is v.

```
pub fn atan(both: single) -> single
```

Arc tangent of a single value. Returns angle in (-PI/2, PI/2).

```
pub fn atan2(both: single, v2: single) -> single
```

Arc tangent of y/x, preserving the correct quadrant. Use instead of atan when you have separate x/y components.

```
pub fn ceil(both: single) -> single
```

Round up to the nearest integer value. Use to compute required buffer sizes from fractional counts.

```
pub fn floor(both: single) -> single
```

Round down to the nearest integer value. Use to convert a float position to a tile index.

```
pub fn round(both: single) -> single
```

Round to the nearest integer value (half rounds away from zero).

```
pub fn sqrt(both: single) -> single
```

Square root. Use for distances and normalization.

```
pub fn log(both: single, v2: single) -> single
```

Logarithm of v in the given base. Use for converting between scales (e.g., decibels).

```
pub fn pow(both: single, v2: single) -> single
```

Raises base to the power exp. Use for exponential growth curves and scaling.

```
pub fn abs(both: float) -> float
```

Absolute value for double-precision floats.

```
pub PI = OpMathPiFloat()
```

The ratio of a circle's circumference to its diameter (3.14159...).

```
pub E = OpMathEFloat()
```

Euler's number, the base of natural logarithms (2.71828...).

```
pub fn cos(both: float) -> float
```

Cosine. Use for circular motion: x = r * cos(angle).

```
pub fn sin(both: float) -> float
```

Sine. Use for circular motion: y = r * sin(angle).

```
pub fn tan(both: float) -> float
```

Tangent. Use for slopes and perspective projection.

```
pub fn acos(both: float) -> float
```

Arc cosine. Returns the angle (in radians) whose cosine is v.

```
pub fn asin(both: float) -> float
```

Arc sine. Returns the angle whose sine is v.

```
pub fn atan(both: float) -> float
```

Arc tangent of a single value. Returns angle in (-PI/2, PI/2).

```
pub fn atan2(both: float, v2: float) -> float
```

Arc tangent of y/x, preserving the correct quadrant. Use instead of atan when you have separate x/y components.

```
pub fn ceil(both: float) -> float
```

Round up to the nearest integer value. Use to compute required buffer sizes from fractional counts.

```
pub fn floor(both: float) -> float
```

Round down to the nearest integer value. Use to convert a float position to a tile index.

```
pub fn round(both: float) -> float
```

Round to the nearest integer value (half rounds away from zero).

```
pub fn sqrt(both: float) -> float
```

Square root. Use for distances and normalization.

```
pub fn log(both: float, v2: float) -> float
```

Logarithm of v in the given base. Use for converting between scales (e.g., decibels).

```
pub fn pow(both: float, v2: float) -> float
```

Raises base to the power exp. Use for exponential growth curves and scaling.

23.3. Text

Functions for working with text (UTF-8 strings) and character values.

```
pub fn len(both: text) -> integer
```

Number of bytes in the text. Use for bounds checks and iteration limits.

```
pub fn len(both: character) -> integer
```

Byte length of the character's UTF-8 encoding (1–4).

```
pub fn split(self: text, separator: character) -> vector<text>
```

Splits self on every occurrence of separator and returns the parts as a vector. Use to parse CSV lines or space-separated tokens.

Functions for searching, transforming, and classifying text and character values. Character classification functions return true only if every character in the text satisfies the condition. The single-character variants test one code point.

```
pub fn starts_with(self: text, value: text) -> boolean
```

Returns true if self begins with value. Use for prefix matching (e.g., protocol detection).

```
pub fn ends_with(self: text, value: text) -> boolean
```

Returns true if self ends with value. Use for suffix matching (e.g., file extension checks).

```
pub fn trim(both: text) -> text[both]
```

Removes leading and trailing whitespace. Use when processing user input or file content.

```
pub fn trim_start(self: text) -> text[self]
```

Removes leading whitespace only.

```
pub fn trim_end(self: text) -> text[self]
```

Removes trailing whitespace only.

```
pub fn find(self: text, value: text) -> integer
```

Returns the byte index of the first occurrence of value, or null if not found. Use to locate substrings before slicing.

```
pub fn rfind(self: text, value: text) -> integer
```

Returns the byte index of the last occurrence of value, or null if not found. Use to find file extensions or the last path separator.

```
pub fn contains(self: text, value: text) -> boolean
```

Returns true if value appears anywhere in self. Simpler than find when you only need a yes/no answer.

```
pub fn replace(self: text, value: text, with: text) -> text
```

Returns a copy of self with every occurrence of value replaced by with.

```
pub fn to_lowercase(self: text) -> text
```

Returns a lowercase copy. Use for case-insensitive comparisons.

```
pub fn to_uppercase(self: text) -> text
```

Returns an uppercase copy.

```
pub fn is_lowercase(self: text) -> boolean
```

True if all characters are lowercase letters.

```
pub fn is_lowercase(self: character) -> boolean
```

True if the character is a lowercase letter.

```
pub fn is_uppercase(self: text) -> boolean
```

True if all characters are uppercase letters.

```
pub fn is_uppercase(self: character) -> boolean
```

True if the character is an uppercase letter.

```
pub fn is_numeric(self: text) -> boolean
```

True if all characters are numeric digits (Unicode numeric, not just ASCII 0–9).

```
pub fn is_numeric(self: character) -> boolean
```

True if the character is a numeric digit.

```
pub fn is_alphanumeric(self: text) -> boolean
```

True if all characters are letters or digits. Use to validate identifiers or tokens.

```
pub fn is_alphanumeric(self: character) -> boolean
```

True if the character is a letter or digit.

```
pub fn is_alphabetic(self: text) -> boolean
```

True if all characters are alphabetic.

```
pub fn is_alphabetic(self: character) -> boolean
```

True if the character is alphabetic.

```
pub fn is_whitespace(self: text) -> boolean
```

True if all characters are whitespace. Use to detect blank lines.

```
pub fn is_control(self: text) -> boolean
```

True if all characters are control characters.

23.4. Collections

Operations on vector<T> — the primary ordered collection type. Vectors are grown by appending with += and elements are accessed by index.

```
pub fn len(both: vector) -> integer
```

Number of elements in the vector. Use in loop bounds: for i in 0..v.len().

23.5. Output and Diagnostics

```
pub fn assert(test: boolean, message: text, file: text, line: integer)
```

Panics with message if test is false. Use to verify invariants during development. In production mode (–production), logs an error instead of aborting. The file and line are injected by the compiler; do not pass them manually.

```
pub fn panic(message: text, file: text, line: integer)
```

Immediately terminates execution with message. Use for unrecoverable error states. In production mode (–production), logs a fatal entry instead of aborting. The file and line are injected by the compiler; do not pass them manually.

```
pub fn log_info(message: text, file: text, line: integer)
```

Write a structured log record at the chosen severity. The file and line are injected by the compiler; do not pass them manually.

```
pub fn log_warn(message: text, file: text, line: integer)
```

```
pub fn log_error(message: text, file: text, line: integer)
```

```
pub fn log_fatal(message: text, file: text, line: integer)
```

```
pub fn print(v1: text)
```

Writes v to standard output without a newline. Use for progress output and building up a line incrementally.

```
pub fn println(v1: text)
```

Writes v followed by a newline. The standard choice for line-oriented output.

23.6. Parallel

23.7. File System

Types and functions for reading and writing files. A File value is obtained via file() and carries the path, format, and an internal reference.

```
pub struct EnvVariable {
    name: text,
    value: text
}
```

An environment variable as a name/value pair.

```
pub struct Pixel {
    r: integer limit(0, 255) not null,
    g: integer limit(0, 255) not null,
    b: integer limit(0, 255) not null
}
```

Data per pixel of the png graph. Fields r, g, b hold red, green and blue intensity, each 0–255.

```
pub struct Image {
    name: text,
    width: integer,
    height: integer,
    data: vector<Pixel>
}
```

Database structure with image data. The actual data is in height rows of width length.

```
pub enum Format {
    TextFile,
    LittleEndian,
    BigEndian,
    Directory,
```

```
    NotExists
}
```

Describes how a file is opened. TextFile: read or write as UTF-8 text (default). LittleEndian: binary mode, least-significant byte first. BigEndian: binary mode, most-significant byte first. Directory: represents a directory path. NotExists: no file or directory exist.

```
pub struct File {
    path: text,
    size: long,
    format: Format,
    ref: i32,
    current: long,
    next: long
}
```

A handle to a filesystem entry. Fields: path (full path), size (file size in bytes), format (open mode), current (byte position after last read), next (byte position to read next).

```
pub fn value(self: Pixel) -> integer
```

Returns the pixel colour as a packed 24-bit integer (0xRRGGBB). Use for fast colour comparison or storage.

```
pub fn content(self: File) -> text
```

Reads the entire file as a UTF-8 text value. Use for small configuration files or scripts.

```
pub fn lines(self: File) -> vector<text>
```

Reads the file and splits it into lines. Use when processing line-by-line (logs, CSV, etc.).

```
pub fn file(path: text) -> File
```

Use as the entry point for all file I/O. Paths outside the project directory are treated as NotExists without touching the filesystem.

```
pub fn exists(path: text) -> boolean
```

Use to check whether a path is accessible before reading or writing it. Paths outside the project directory are always absent.

```
pub fn delete(path: text) -> boolean
```

Use to remove a file after processing or as a cleanup step. Returns false when the file did not exist or is outside the project directory.

```
pub fn move(from: text, to: text) -> boolean
```

Use to rename or relocate a file within the project. Both paths must stay within the project directory; the destination must not already exist.

```
pub fn set_file_size(self: File, size: long) -> boolean
```

Truncate or extend the file to exactly `size` bytes. Truncating removes bytes beyond `size`; extending fills with null bytes. Returns false if the file is a directory, does not exist, or `size` is negative. Any open handle to the file is closed; reopen the file for further I/O.

```
pub fn files(self: File) -> vector<File>
```

Returns the entries inside a directory. The File must have format == Format.Directory. Use to iterate over all files in a folder.

```
pub fn write(self: File, v: text)
```

Writes v as UTF-8 text to the file. Overwrites existing content.

```
pub fn png(self: File) -> Image
```

Decodes a PNG file and returns an Image. Returns null if the file is not in text format. Use to load sprite sheets or textures.

23.8. Environment

```
pub fn env_variables() -> vector<EnvVariable>
```

Returns all environment variables as a vector of EnvVariable records (fields: name, value). Use to inspect or forward the full environment.

```
pub fn env_variable(name: text) -> text
```

Returns the value of the environment variable name, or null if it is not set. Use to read configuration from the shell environment.

Functions for interacting with the host operating system.

```
pub fn arguments() -> vector<text>
```

Returns the command-line arguments passed to the program. The first element is typically the program name.

```
pub fn directory(v: &text = "") -> text
```

Returns the current working directory, optionally with v appended as a subpath. Use to construct absolute paths relative to where the program was launched.

```
pub fn user_directory(v: &text = "") -> text
```

Returns the current user's home directory, optionally with v appended. Use for storing user-specific data or configuration.

```
pub fn program_directory(v: &text = "") -> text
```

Returns the directory containing the running executable, optionally with v appended. Use to locate assets bundled alongside the program.